Lecture 08

Authentication and Authorization

Backend Engineering from First Principles

Large-font study notes • Concepts • Flows • Comparisons • Interview revision

Core distinction
Authentication answers **Who are you?**
Authorization answers **What can you do?**

Prepared from the supplied lecture transcript

1. Authentication vs. Authorization

Authentication (AuthN) establishes the identity of a subject. It answers: **Who are you?**

Authorization (AuthZ) determines what an authenticated subject is allowed to do. It answers: **What can you do?**

Example: logging into a note-taking app establishes identity. Checking whether the user can read, edit or permanently delete a note is authorization.

2. Historical evolution of authentication

Period / stage	Approach	Main idea and limitation
Pre-industrial	Implicit human trust	Community elders vouched for people; handshakes represented agreement. Did not scale beyond familiar circles.
Medieval	Wax seals	Physical identity tokens/signatures on documents; relied on possession and could be forged.
Industrial Revolution	Passphrases / shared secrets	Telegraph operators used pre-agreed phrases: an early “something you know” factor.
Mid-20th century	Digital passwords	Multi-user computers needed user separation. Plaintext password storage exposed serious risks.
Secure storage	Hashing	One-way fixed-length representations replaced plaintext password storage.
1970s	Diffie–Hellman, Kerberos	Key exchange over untrusted media; ticket-based authentication through a trusted third party.
1990s	MFA and biometrics	Multiple factors strengthened password-only systems facing brute-force and dictionary attacks.
21st century	OAuth 2.0, JWT, zero trust, passwordless	Cloud, mobile and APIs required scalable mechanisms.
Emerging	Decentralized identity, behavioral biometrics, post-quantum cryptography	Potential future approaches.

Multi-factor authentication (MFA)

MFA combines different factor categories: **something you know** (password/PIN), **something you have** (smart card/hardware token/OTP generator), and **something you are** (fingerprint/retina pattern). Biometrics can face false positives, false negatives and template-security challenges.

Hashing

A hash function maps input to a fixed-length representation. The same input under the same algorithm produces the same output. Passwords should not be stored in plaintext. Use dedicated password-hashing functions with unique salts; a fast general-purpose hash alone is not suitable.

3. Sessions, JWTs and Cookies

HTTP is stateless

HTTP treats each request as an independent interaction. Dynamic applications need continuity—for example, shopping carts and persistent login—so sessions provide server-side user context.

Sessions

A **session** is temporary server-side context associated with a user. On login, the server verifies credentials, creates a unique session ID, stores user/session data, and sends the ID to the browser (commonly in a cookie). Later requests return the cookie; the server looks up the session and uses its data. Sessions expire according to policy.

Storage evolution in the lecture: file-based sessions (simple but hard to scale), database-backed sessions (persistent across restarts), and distributed/in-memory stores such as Redis or Memcached (fast lookups across servers).

JWT (JSON Web Token)

A JWT is a compact signed token that carries claims. Its three parts are **header.payload.signature**. Header: metadata such as signing algorithm. Payload: claims such as `sub` (subject), `iat` (issued-at), and role. Signature: verifies issuer authenticity and token integrity.

JWTs support stateless verification, distributed service scalability and portability. Trade-offs include token theft/impersonation and difficult immediate revocation. A hybrid approach can check a server-side denylist after signature verification, but adds a lookup.

> A signed JWT is not automatically encrypted. Its payload is normally readable; do not put secrets in it.

Cookies

A cookie is browser-stored data set by a server and sent on later requests to the relevant site. It may carry a session ID or token. Important attributes: **HttpOnly** (not readable by ordinary JavaScript), **Secure** (HTTPS only), **SameSite** (cross-site sending policy), plus Domain, Path and expiry.

4. Stateful and Stateless Authentication

Stateful flow

```
Browser -- username/password --> Backend -- verify credentials\nBackend -- create session +
user data --> Session Store (e.g. Redis)\nBrowser <-- session cookie -------- Backend\nBrowser
-- later request + cookie --> Backend -- lookup session --> Store\nBackend <-- session/user
data ------------------------------ Store
```

Advantages: centralized control, visibility into active sessions, easier logout and revocation. Challenges: shared session-store dependency, distributed coordination, operational complexity and lookup latency.

Stateless flow

```
Browser -- username/password --> Backend -- verify credentials\nBackend -- create + sign JWT
--> Browser\nBrowser -- later request + JWT --> Backend -- verify signature/read
claims\nBrowser <-- response ------------- Backend
```

Advantages: no per-user session-store lookup in the basic flow, scalability across distributed services, useful for APIs/mobile clients. Challenges: harder revocation, stolen-token risk, and careful expiry/key

management.

Choosing between them

Requirement	Often a good fit
Browser app with strict session control and easy revocation	Stateful sessions
Distributed APIs needing independent token verification	Stateless tokens
Mobile / third-party API clients	Often stateless, architecture dependent
Need both browser control and scalable API access	Deliberate hybrid design

5. API-key authentication

An **API key** is a cryptographically random secret issued to a developer/application for programmatic API access. Generate it, store it in a secret manager or protected environment variable, and send it with API requests (commonly in a header). The provider checks identity, permissions, limits or expiry.

Best suited to machine-to-machine communication, server-to-server calls, automation and single-purpose API access. Keys are simple and avoid interactive login for every machine request.

> Treat API keys like passwords: never commit them to Git, expose them in frontend code or log them. Revoke leaked keys and use minimum permissions.

6. OAuth 2.0 and OpenID Connect

Delegation problem

An application may need limited access to resources on another platform (for example, importing contacts). Sharing a password gives broad access and makes revocation difficult. Delegation grants limited access without sharing the password.

OAuth 2.0

OAuth 2.0 is an **authorization framework** for delegated access. Roles: **resource owner** (user/data owner), **client** (requesting app), **authorization server** (issues tokens after authorization), and **resource server** (hosts protected API/data).

```
1. Client redirects user to authorization server.\n2. User authenticates and grants requested consent/scopes.\n3. Authorization server issues an access token to the client.\n4. Client presents token to resource server.\n5. Resource server allows only the permitted access.
```

Flow names in the lecture: Authorization Code (server-side/web apps), Implicit (historical browser flow, now discouraged), Client Credentials (machine-to-machine), and Device Authorization (limited-input devices such as smart TVs).

OpenID Connect (OIDC)

OIDC is an identity layer on top of OAuth 2.0. It introduces an **ID token**, commonly a JWT, with identity claims such as subject, issuer, audience and issue/expiry times. It supports “Sign in with Google” style login.

> OAuth 2.0: “Can this app access this resource?” OIDC: “Who is this user?”

7. Authorization and RBAC

Role-Based Access Control

RBAC groups permissions into roles and assigns roles to users. For a notes app, a viewer may read, an editor may read/write, and an admin may manage users or access restricted notes.

```
User signs up -> role assigned\nUser authenticates -> session ID / token sent\nBackend
identifies user and role\nAuthorization middleware checks requested action\nAllowed -> continue
| Not allowed -> reject (403)
```

The role can be obtained from trusted token claims or a session/database lookup. Authorization should check the required action/resource, not merely whether the user is logged in.

401 Unauthorized: authentication is missing or invalid. **403 Forbidden:** caller is authenticated but lacks permission.

8. Authentication security considerations

Generic authentication errors

Messages such as “user not found” or “incorrect password” can reveal whether an account exists. Use generic credential-failure messages such as **“Authentication failed”** to reduce account enumeration.

Timing attacks

If nonexistent usernames fail immediately but existing usernames trigger slower password-hash verification, response-time differences can reveal account existence.

Mitigations described: use constant-time comparison functions for secret/hash comparisons and make failure paths more uniform. Comparable work for nonexistent users can reduce observable timing differences.

9. Quick comparison

Method	State / validation	Typical use
Stateful session	Server-side session lookup	Browser/web sessions
Stateless JWT	Signature/claims verification; no lookup in basic flow	Distributed APIs/services
API key	Provider validates key and its metadata	Machine-to-machine API access
OAuth 2.0	Access token and scopes	Delegated authorization
OIDC	ID token validation with identity provider	Federated login
RBAC	Role/permission check	Authorization after identity is known

10. Interview questions

1. Authentication vs authorization? Authentication verifies identity; authorization checks permissions.

2. Why is HTTP stateless? Each request is independent; HTTP does not automatically retain application state.

3. What is a session? Server-side user context referenced by a session ID.

4. What is a cookie? Browser-stored data that can accompany later requests to a site.

5. JWT parts? Header, payload, signature.

6. Is a JWT payload encrypted? Not by default; a signed JWT is usually readable.

7. JWT drawback? Revocation is difficult before expiry; stolen tokens can be abused.

8. How can JWT revocation be supported? Short expiry, refresh-token strategy or a server-side denylist/revocation check.

9. API key use? Programmatic and machine-to-machine access.

10. What does OAuth solve? Delegated authorization without sharing the user's password.

11. OAuth vs OIDC? OAuth is authorization; OIDC adds authentication/identity.

12. What is RBAC? Permissions are grouped into roles assigned to users.

13. 401 vs 403? 401: missing/invalid authentication; 403: authenticated but not permitted.

14. Timing attack? Inferring information from differences in response/operation time.

15. Why generic login errors? To avoid revealing whether an account exists or which check failed.

11. Last-minute revision sheet

AuthN = Who are you? AuthZ = What can you do?

HTTP is stateless; sessions add server-side continuity. Session ID references server-stored data. Cookie is a browser storage/transport mechanism, not an authentication protocol by itself.

JWT = header.payload.signature. Signed does not mean encrypted. Stateful = session lookup and easier revocation. Stateless = scalable token verification and harder revocation.

API key = machine-to-machine access. OAuth 2.0 = delegated authorization. OIDC = identity layer on OAuth 2.0. RBAC = roles map to permissions.

401 = unauthenticated/invalid. 403 = authenticated but not allowed. Generic login errors reduce account enumeration. Timing differences can leak account existence.

12. Implementation cautions

Passwords should be hashed, not reversibly encrypted. Use a purpose-built password-hashing scheme with salts.

Configure JWT signing and verification safely; do not blindly trust an algorithm named in an untrusted token header. JWT payloads are normally readable.

For browser cookies, consider HttpOnly, Secure and SameSite. Cookies are automatically attached, so CSRF protection may be needed. Local storage has different XSS risks.

OAuth access tokens and OIDC ID tokens have different purposes. An access token is for protected resources; an ID token tells the client about the authentication event.

The lecture recommends a reputable external authentication provider for production systems unless you are confident in implementing the complete security lifecycle.

> End of Lecture 08 notes. The supplied transcript ends by introducing the next topic: validations and transformations.